

One-step mobile-network traffic forecasting in Milan

Christian Tonny | ML Techniques I | Formative Assignment 1

Demo video: PASTE_VIDEO_LINK_HERE

Code: <https://github.com/irachrist1/ml-techniques-1-formative-1>

1. Introduction

Network operators have to decide where to put capacity before demand arrives, so a forecast of the next few minutes of traffic has a practical use. This report asks a narrow version of that question: given only an area's own recent history, how well can three different sequential models predict its Internet activity ten minutes ahead, and does the answer change from one area to another?

The value being predicted is the dataset's Internet activity count for the next 10-minute interval. It is not bandwidth and not a byte count, so the error figures below are in activity units and cannot be read as megabytes.

I process the whole November-December 2013 Milan collection, rank all 10,000 areas by total activity, and then compare a regularized linear autoregression, an LSTM and a causal dilated CNN on the busiest areas. Persistence and a daily-seasonal forecast are included so that any gain from the more complex models has something to beat. December 16-22 is held back and is never used to pick a model or a hyperparameter.

2. Related work and model motivation

Barlacchi et al. [1] describe this dataset and document its daily, weekly and spatial variation. That is the reason I check temporal dependence per area instead of assuming one shared pattern, and the reason I do not attach land-use labels to square IDs. The paper's well-known Bocconi and Navigli examples are squares 4259 and 4456, not the 4159 and 4556 this assignment asks for, so no interpretation transfers.

Azari et al. [2] compare LSTM and ARIMA on cellular packet traffic and find that whether complexity pays depends on the traffic regime, the features available and how much training data there is. That is what pushed me to keep a simple statistical comparator alongside the neural models rather than assuming the neural models win. My RidgeAR is not their ARIMA: it has no moving-average error terms and no differencing operator, it is a penalized linear fit on lagged values.

Bai et al. [3] argue that causal dilated convolutions can replace recurrence for sequence modeling. My CausalCNN uses that mechanism but not their full residual TCN, so their benchmark rankings should not be expected to carry over. Zhang and Patras [4] use spatial structure and longer horizons; that work sets out what my single-area, one-step setup is missing rather than giving me a number to compare against.

Headline result: on area 5161, the busiest area, the best learned model at the reference seed is CausalCNN with RMSE 118.16, against persistence at 134.88. Section 8 shows why I do not treat that as a general ranking.

3. Dataset, preparation and memory management

Each daily file from the publisher holds square ID, interval timestamp, country code, SMS in/out, call in/out and Internet activity. The 61 files I need total 20.48 GB and 314,966,126 rows. January 1 exists in the collection but falls outside the period, so it is excluded. Every file is checked against the publisher's MD5 before anything reads it, and timestamps are validated against a 10-minute grid using Europe/Rome calendar boundaries.

Internet activity is summed over the country-code rows for each square and interval. An empty Internet field is not an observed zero: a bin counts as missing only when no finite value exists for it at all. That leaves 8,784 intervals per area, with 145,354 of the 87,840,000 square-time bins missing. One caveat I cannot resolve from this data alone is that a bin can be observed but only partially covered across country codes, and there is no way to tell that apart from complete coverage.

The loader reads each file in chunks, keeps only square, timestamp and Internet, stores square IDs as int32 and leaves activity as float64. The accumulators are fixed-size sum and count arrays of 144 by 10,000, which is 17.28 MB regardless of how big the file is. Working one day at a time is what keeps the 20.48 GB collection off the heap. Checksums and date validation also stop a day being ingested twice, though they say nothing about whether the original measurements are right.

Full-day method	Peak process RAM (MB)	Wall time (s)	Largest DataFrame (MB)
baseline	917.8	2.23	309.9
chunked	124.6	1.60	2.0

Both methods run in their own process on the same complete November 1 file, and peak RSS drops by 86.4%. The comparison is only worth anything if the outputs match, so the benchmark asserts that: the missing masks are identical, the observation counts are identical, and the largest difference between the two aggregates is 1.8e-12, which is summation order. The wall times are a single run each with an uncontrolled filesystem cache, so I would not quote the speedup. What this does trade away is disk and repeated I/O, since the raw files stay where they are.

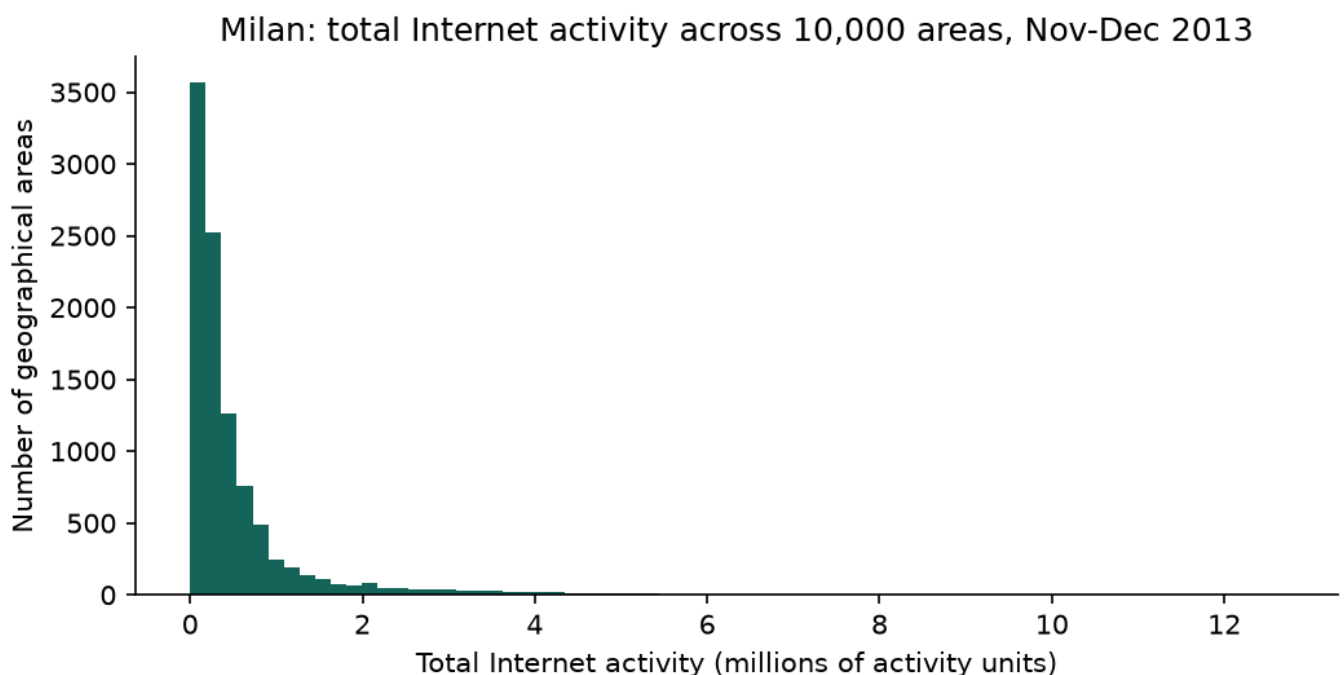


Figure 1. Total observed Internet activity by area over November-December. Totals exclude missing values, and coverage is audited separately.

The distribution has a long right tail. Median total activity is 274,706, the 99th percentile is 4,666,202, and the maximum is 12,682,673. The top three squares are **5161, 5059, 5259**. They hold 0.62% of observed activity and the top 1% of areas hold 11.09%.

Because totals only count observed bins, a badly covered area is understated and could in principle be ranked too low. 98 areas have less than full coverage and the worst sits at 8.4%. If I scale every one of them up by 1/coverage, which is the most generous correction available, the largest becomes 891,902, still well below the 10,436,792 needed to reach third place. So missingness did not change who is in the top three. Ranking on full-period totals does mean the

selection used the evaluation week's activity, which is a selection dependence worth naming even though it never touches model fitting.

4. Exploratory and temporal analysis

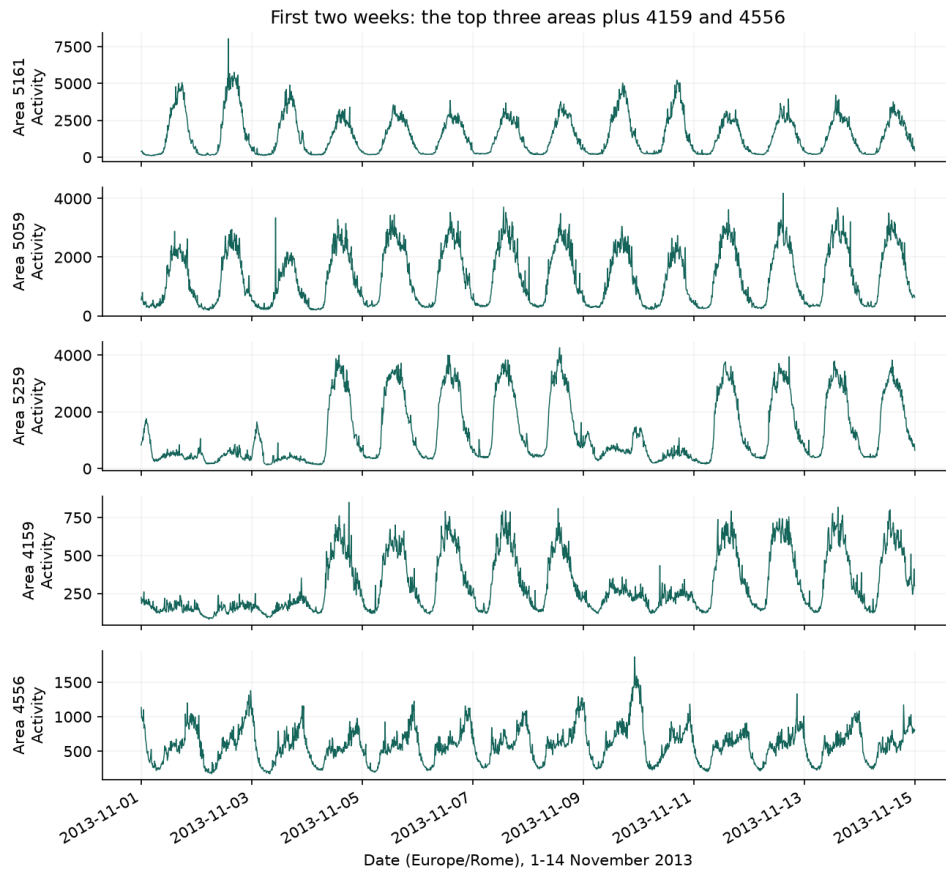


Figure 2. First two weeks for the three busiest areas plus squares 4159 and 4556. Each panel keeps its own scale, so compare shape as well as level.

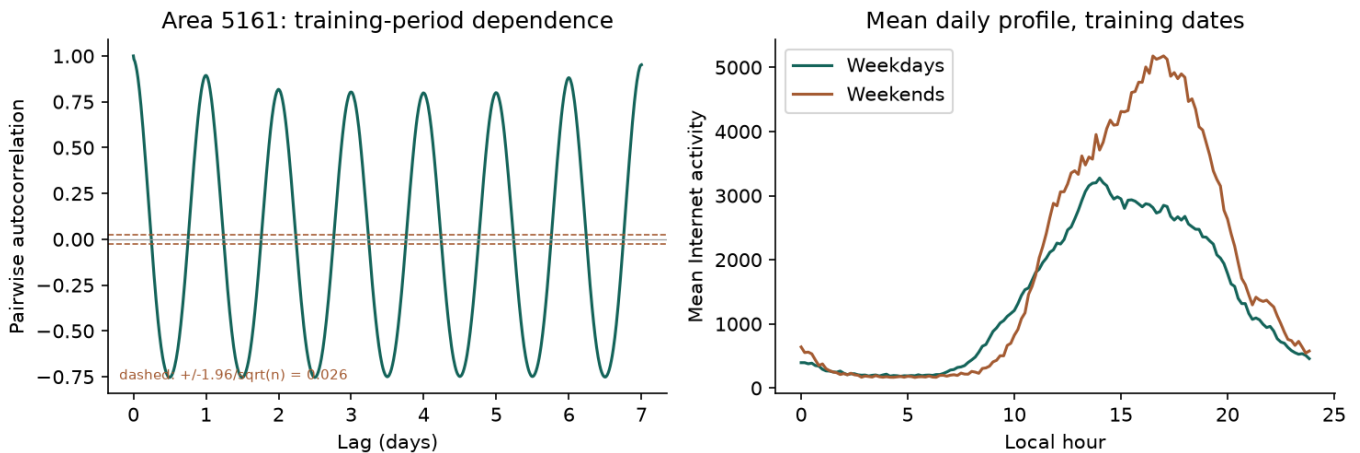


Figure 3. Training-period autocorrelation and weekday/weekend daily profiles for area 5161. The dashed lines are the $\pm 1.96/\sqrt{n}$ white-noise band.

The five series differ in more than scale. Areas 5259 and 4159 sit low through the first weekend and then jump to a weekday plateau, and they drop again on 9-10 November. Area 5161 runs the other way, with its highest peaks at the weekend. Area 5059 holds a broad daytime plateau with short bursts, and 4556 is the noisiest relative to its own mean and carries isolated one-interval spikes.

The weekday/weekend split is measurable, not just visible. Over the training dates, area 5161 averages 1847 at weekends against 1357 on weekdays, a ratio of 1.36, while area 5259 is at 0.42 and 4159 at 0.54. One calendar detail matters for reading the left edge of Figure 2: 1 November is All Saints' Day, a public holiday in Italy, and it fell on a Friday in 2013. So the first low stretch is a three-day holiday weekend rather than an ordinary one, and the 9-10 November weekend is the cleaner comparison. These are activity schedules; I cannot tell from this data what land use produces them or what caused any individual spike.

For area 5161, correlation with the previous interval is 0.982, with the same time yesterday 0.894, and with the same time last week 0.952. All three are far above the 0.026 white-noise band, so none of them is in doubt. The interesting part is the gap between them: the 10-minute lag is much stronger than the daily lag, which is a direct argument for giving the models recent history rather than relying on the daily shape. Correlation at a six-hour lag is -0.017, effectively nothing, which is the half-day trough of the daily cycle.

4.1 Statistical characterization

Area	Training mean	Training CV	Lag-1 corr.	Daily-lag corr.	Weekend/weekd ay	Missing bins
5161	1511.4	0.92	0.982	0.894	1.36	0
5059	1342.6	0.71	0.972	0.912	0.81	0
5259	1314.1	0.85	0.989	0.669	0.42	0
4159	317.2	0.59	0.969	0.743	0.54	0
4556	582.4	0.43	0.943	0.769	1.14	0

CV is standard deviation over mean. The areas differ in level, in variability and in how much of their structure is daily rather than immediate, which is why I fit and score them separately. Note that a strong daily correlation does not mean yesterday's value is a good 10-minute forecast, and Section 7 shows exactly that.

On the longest fully observed training stretch for area 5161 (5,472 points) the ADF statistic is -4.103 with $p = 9.57e-04$, using 152 lags chosen by AIC from a ceiling of 168. I set that ceiling above 144 on purpose, because a lag window shorter than one day cannot span the seasonal cycle the test is being asked to look through. After first differencing, $p = 3e-15$. The unit-root null is rejected, but that is all it is: the series plainly has a daily cycle and shifting levels, and ADF does not contradict either. A robust STL with period 144 puts the daily seasonal strength at 0.870, and 0 training points needed interpolation for that decomposition, which is descriptive only and never feeds a model.

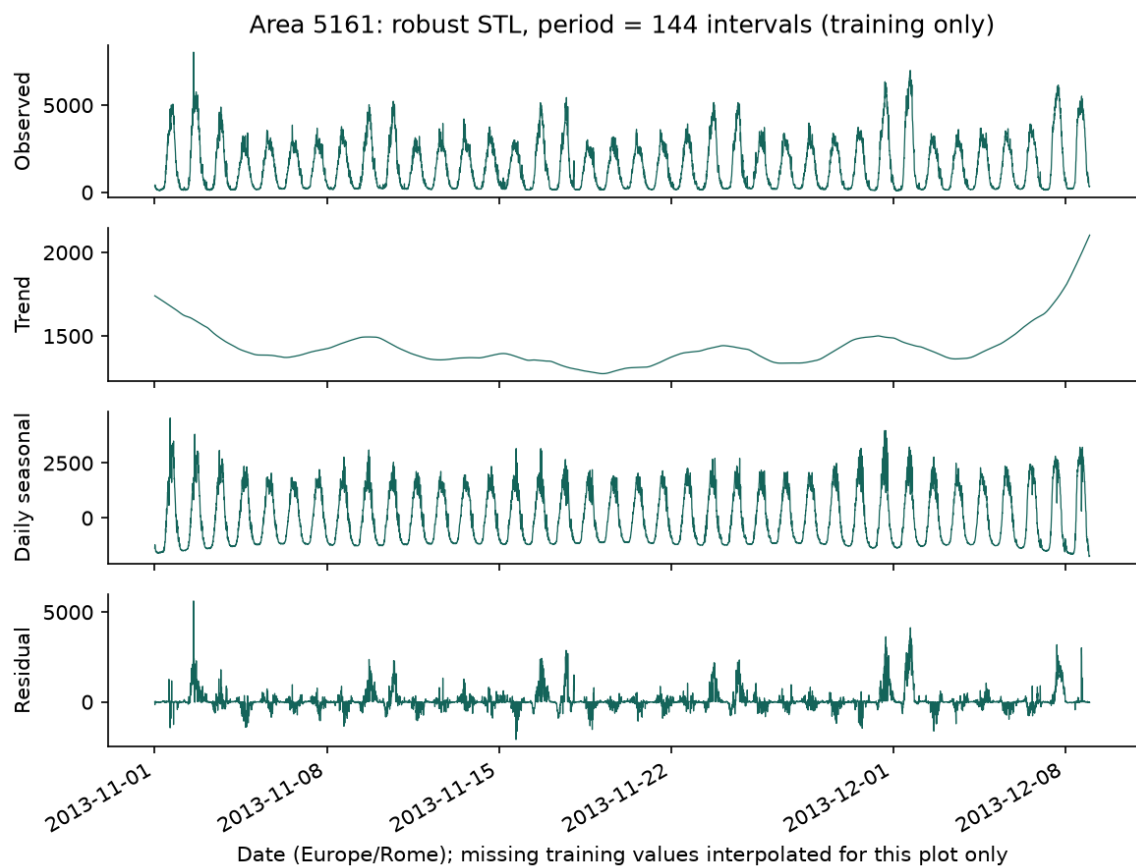


Figure 4. Robust STL of the training period for area 5161. The daily component dominates and the trend moves slowly, which is the pattern the models have to exploit.

5. Methodology

To predict $x(t+1)$ a model sees only $x(t-L+1)$ through $x(t)$. Every candidate is required to have a fully observed 144-step history before a timestamp counts as eligible, even when its own lookback is shorter. That is deliberate: without it a short-lookback model would qualify on more timestamps and could win by being scored on an easier subset. Windows with a missing value or a time gap are dropped and counted, and no target is ever imputed.

Split	Local dates	Purpose
Training	Nov 1 - Dec 8	Weights and normalization
Validation	Dec 9 - 15	Tuning and early stopping
Test	Dec 16 - 22	Frozen evaluation

Each area's mean and standard deviation come from its training values only. What the models actually learn is the standardized change, $(x(t+1) - x(t))/\sigma$, and the forecast is $x(t)$ plus σ times the predicted change, clipped at zero. Every model therefore starts from persistence and only has to learn the correction. This is worth flagging early because it shapes the failure mode in Section 8.3: if the next interval is an isolated spike, the correction that minimizes squared error over the training set is close to zero, so the miss is built into the parameterization rather than being a tuning problem.

RidgeAR flattens the normalized history into L lag features and minimizes squared error plus an L2 penalty, with a fitted intercept. The LSTM reads the L -by-1 sequence with tanh state activation and sigmoid gates and passes its final hidden state to a linear scalar head, with no dropout. CausalCNN stacks kernel-3 causal ReLU convolutions with increasing dilation and reads the last time position through a linear head.

Selected model	Lookback (steps)	Architecture / penalty
RidgeAR	144	$\alpha=10$; 145 parameters
LSTM	144	16 gated units; Adam $\text{lr}=0.001$; max epochs=80; patience=4; 1169 parameters
CausalCNN	144	16 filters; dilations 1,2,4,8,16,32,64; Adam $\text{lr}=0.001$; max epochs=80; patience=4; 4785 parameters

The CNN's receptive field is $1 + 2 \times \sum(\text{dilations}) = 255$ steps, which is more than the 144-step input it is given. The last dilation stages therefore reach past the start of the window and add parameters without adding history, and trimming them would be the first thing I would try next.

The neural output heads are initialized at zero. Adam uses gradient-norm clipping at 1, batch size 128, and seeded shuffling of the training windows. Shuffling here is safe because each window is built causally before shuffling, so reordering independent examples cannot move information backwards in time. Validation MSE drives early stopping, which restores the best weights. Each area gives 5,328 training targets and 1,008 validation targets after the common-history rule. Weights and normalization are fitted per area. The test-week predictions are rolling one-step forecasts that use observations already available up to the previous interval; they are not a seven-day recursive forecast from a single origin.

6. Iterative experimentation

All tuning happened on area 5161's validation week. Each round was specified after reading the previous round's results, and the reasoning for every change is stored in the `configs/*.json` file that produced it, with the consolidated record in `results/tuning_log.csv`. Test data was never scored during tuning.

Experiment	Model	Validation RMSE	Epochs run	Training
initial	RidgeAR	159.07	0	0.0
initial	LSTM	163.30	13	4.1
initial	CausalCNN	161.02	20	4.5
daily_history	RidgeAR	153.13	0	0.1
daily_history	LSTM	160.26	19	16.9
daily_history	CausalCNN	160.28	16	10.2
capacity	RidgeAR	152.39	0	0.0
capacity	LSTM	162.04	9	13.0
capacity	CausalCNN	160.09	11	9.1
budget	RidgeAR	152.39	0	0.0
budget	LSTM	160.26	19	17.4
budget	CausalCNN	160.09	11	9.3

Round 1 used six hours of history. Round 2 extended it to a full day, which improved all three models, so I kept it. For the CNN I also had to extend the dilations to cover the longer window, so that round changes two things at once for that model and is not a clean single-variable comparison. Round 3 tested capacity: raising RidgeAR's alpha from 1 to 10 helped slightly, doubling the LSTM to 32 units made it worse so I kept 16, and doubling the CNN to 16 filters helped slightly so I kept that.

Round 4 came out of an audit rather than a hunch. Checking `epochs_run` against the epoch cap in the completed fits showed the cap, not the patience rule, was ending most neural training. On area 5161 the models stop on their own inside 20 epochs, which is exactly why three rounds of tuning on that area never exposed the problem. Raising the cap to 80 leaves area 5161's validation RMSE unchanged and lets the other areas train until early stopping actually fires. After the change, 30 of 30 neural fits stop early and the longest runs 55 epochs. I am treating this as fixing an arbitrary constraint rather than as tuning, since a budget that cuts training off before the stopping rule triggers is not a considered choice.

One thing the table shows that I should not gloss over: RidgeAR has the lowest validation RMSE in every single round. Section 8 returns to this, because it does not match what happens on the test week.

The selected configuration is shared across all five areas while weights and normalization are fitted per area, so this tests whether hyperparameters transfer, not what each area's best settings would be. Final neural fits use seeds 42, 43 and 44, with 42 fixed in advance as the plotting and reference run rather than picked afterwards. RidgeAR is deterministic, so repeating it only varies the timing.

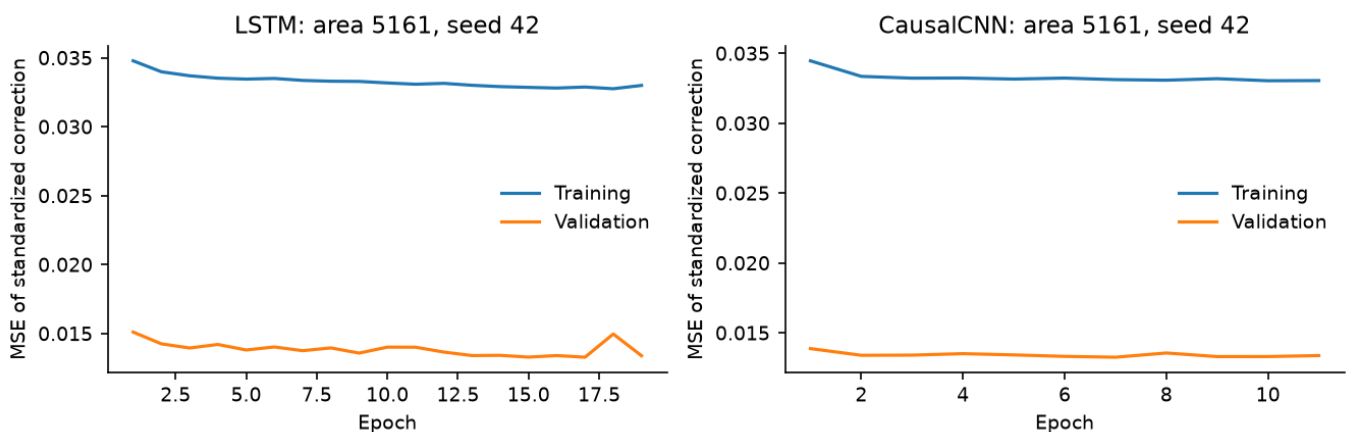


Figure 5. Training and validation loss for the reference fits on area 5161. Early stopping watches validation loss; test data never influences the number of epochs.

Model selection leans on validation RMSE because large misses are what matter for capacity decisions, with MAE and MAPE as secondary evidence. All errors are converted back to activity units before being reported.

7. Results and comparison

Area 5161

Model (seed 42)	MAE	RMSE	MAPE (%)
RidgeAR	85.08	124.18	9.80
LSTM	81.13	118.42	9.09
CausalCNN	79.96	118.16	8.36
Persistence	92.80	134.88	9.19
Daily seasonal	338.59	619.04	25.94

Area 5059

Model (seed 42)	MAE	RMSE	MAPE (%)
RidgeAR	71.79	99.66	7.91
LSTM	75.46	106.22	7.78
CausalCNN	73.96	103.49	7.61
Persistence	81.52	114.38	7.96
Daily seasonal	171.74	245.87	18.02

Area 5259

Model (seed 42)	MAE	RMSE	MAPE (%)
RidgeAR	65.86	93.17	7.74
LSTM	66.84	95.02	7.82
CausalCNN	66.64	93.61	8.21
Persistence	75.97	109.58	8.11
Daily seasonal	470.32	861.62	71.62

MAE is the mean absolute error, RMSE is the square root of mean squared error, and MAPE is 100 times the mean of $|\text{prediction} - \text{actual}| / |\text{actual}|$ over nonzero targets. MAPE excludes zero targets and the excluded count is written to the CSV; no epsilon is substituted. MAE and RMSE use every eligible target. The test week has 1,008 intervals and all of them are scored for every area (5161: 1008, 5059: 1008, 5259: 1008, 4159: 1008, 4556: 1008).

Model	Training (s)	Full test batch (ms)	Single forecast (ms)
CausalCNN	17.09	30.70	0.298
LSTM	23.13	34.59	2.067
RidgeAR	0.05	0.10	0.047

Timing is averaged over five areas and three fits per model on an Apple M2 Pro with 16 GB RAM, CPU only, four TensorFlow intra-op threads. The two scopes are not identical and should not be compared as if they were: neural training time includes building and compiling the graph and running validation each epoch, while the Ridge figure is the solve alone. Inference is measured warm and synchronously, as the median of five full-batch calls and of 30 single-sample calls per fit. It covers the forward pass only, so window construction, scaling and reconstructing the forecast are excluded for all three models.

Forecasts for area 5161

These are the three required model comparisons for this area. All three use the same observed targets and the same reference seed, and each prediction uses only observations available up to the previous interval.

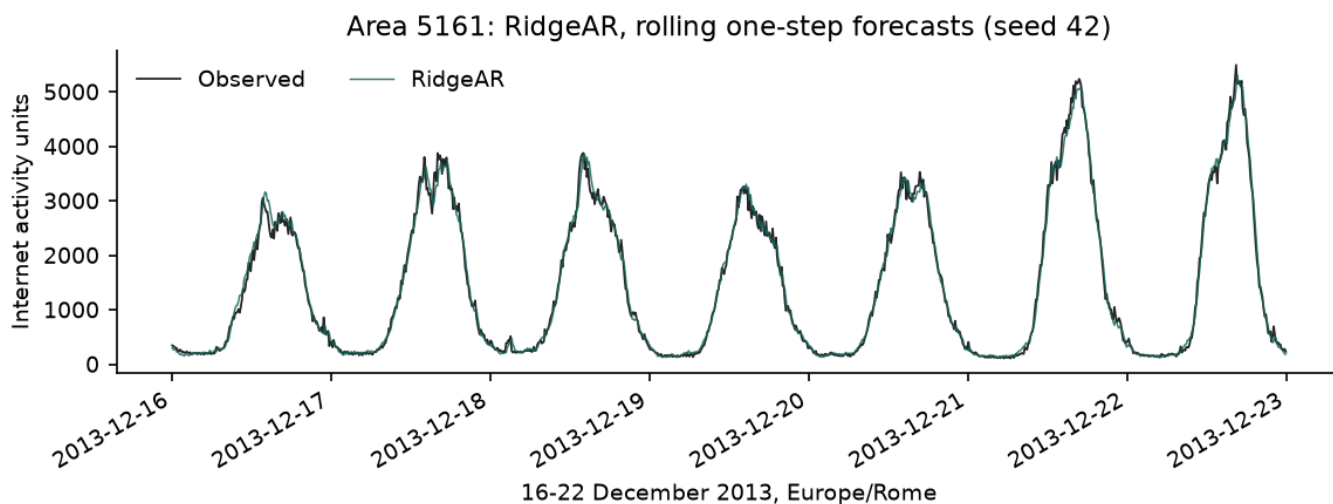


Figure 6a. RidgeAR, area 5161.

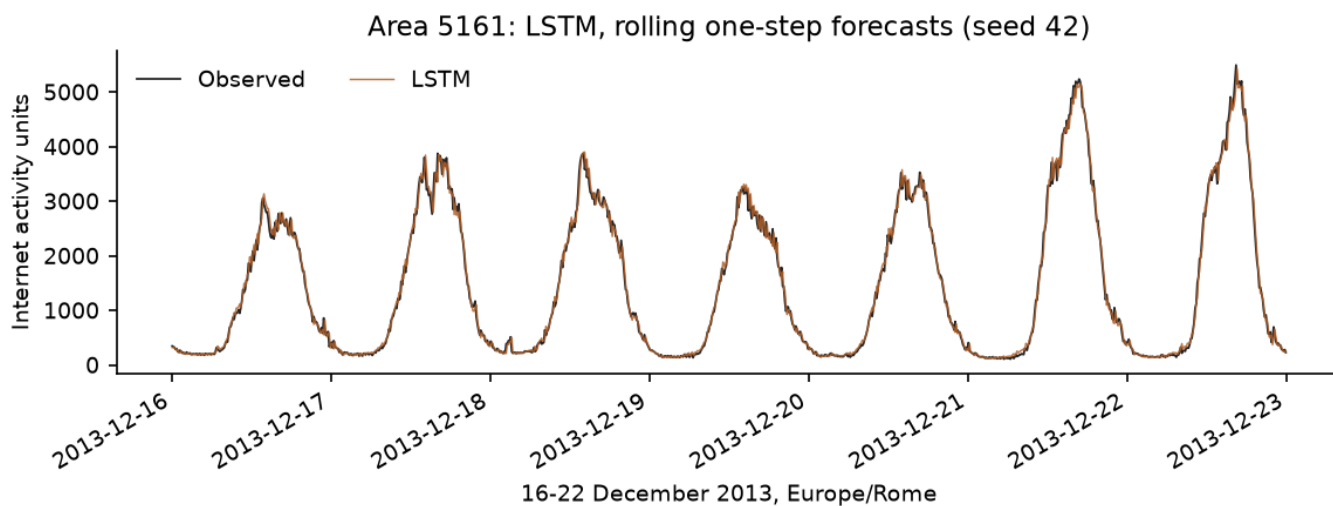


Figure 6b. LSTM, area 5161.

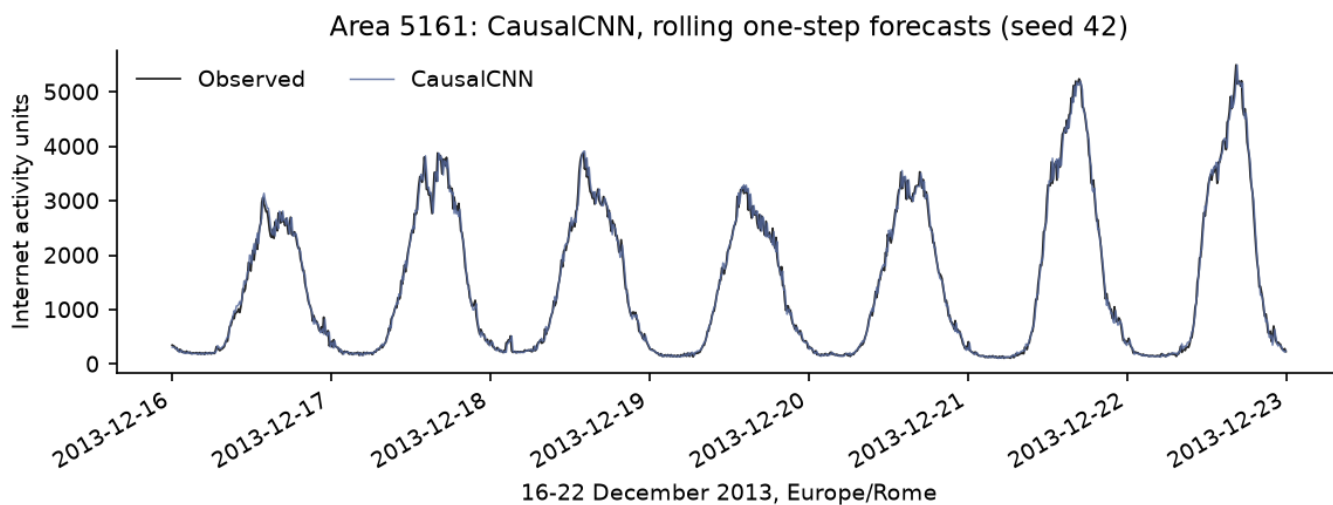


Figure 6c. CausalCNN, area 5161.

Forecasts for area 5059

These are the three required model comparisons for this area. All three use the same observed targets and the same reference seed, and each prediction uses only observations available up to the previous interval.

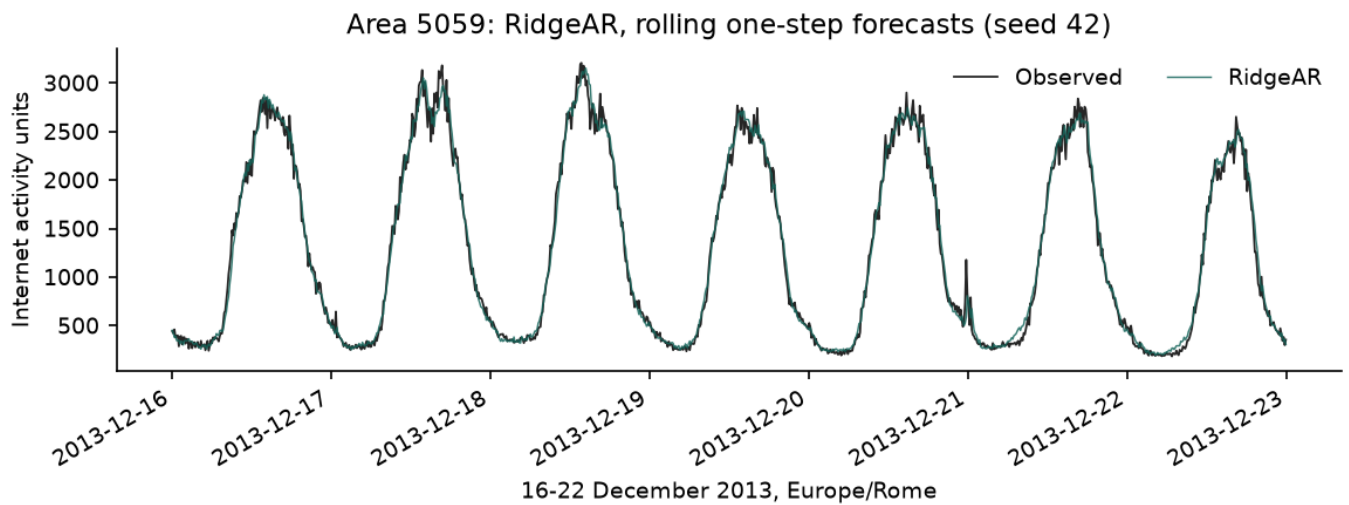


Figure 7a. RidgeAR, area 5059.

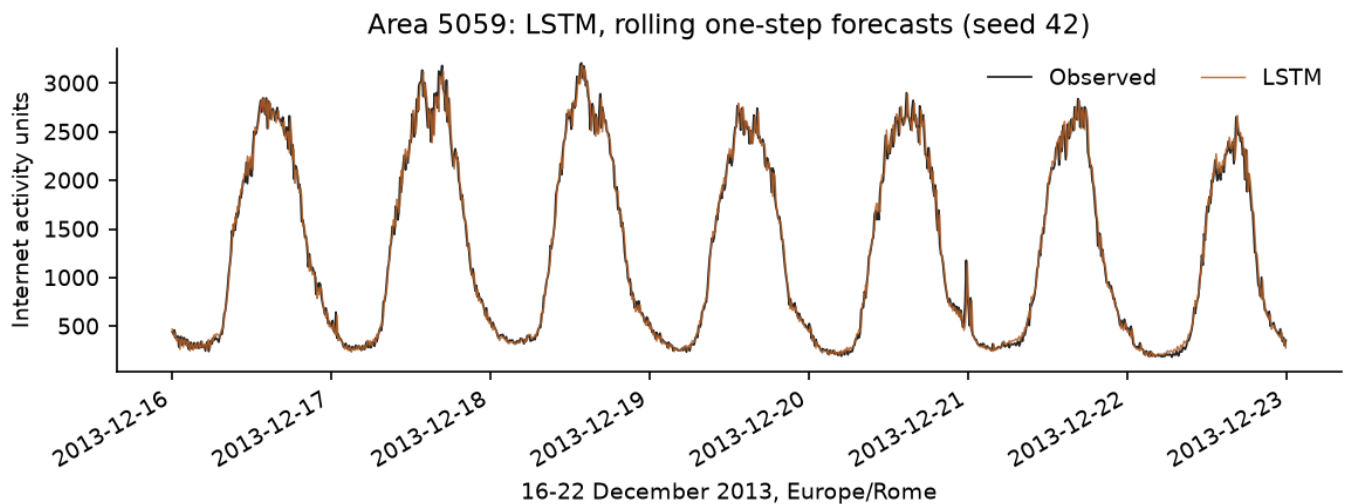


Figure 7b. LSTM, area 5059.

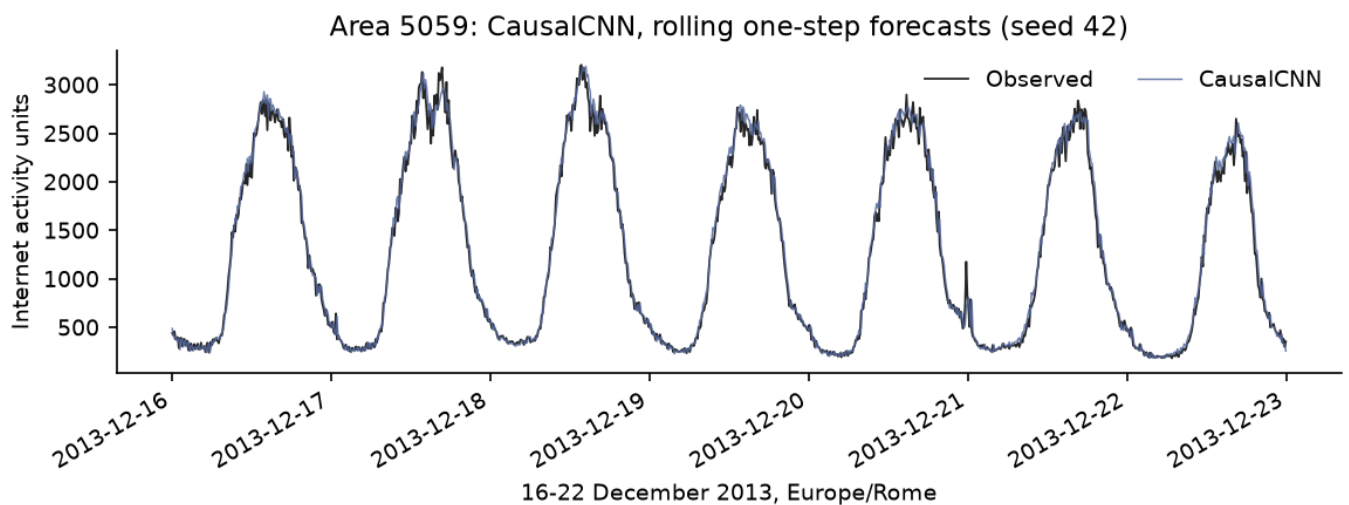


Figure 7c. CausalCNN, area 5059.

Forecasts for area 5259

These are the three required model comparisons for this area. All three use the same observed targets and the same reference seed, and each prediction uses only observations available up to the previous interval.

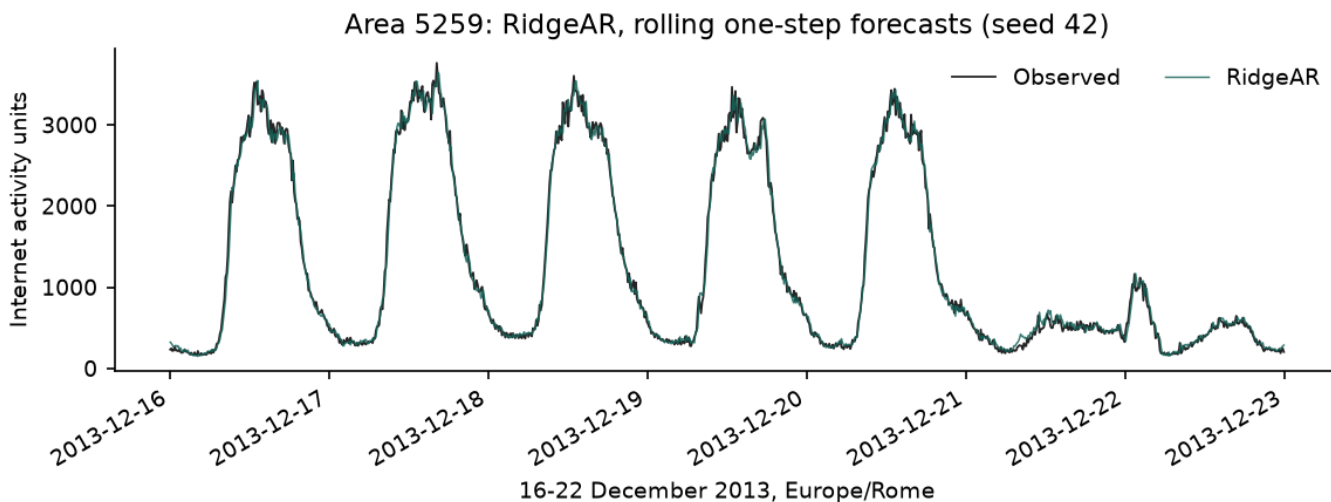


Figure 8a. RidgeAR, area 5259.

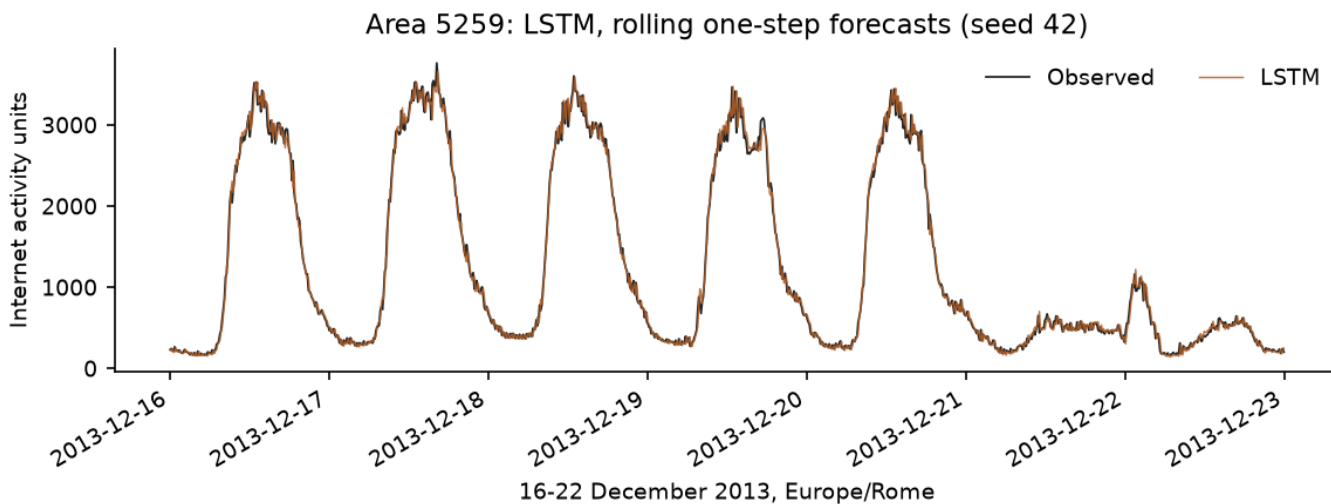


Figure 8b. LSTM, area 5259.

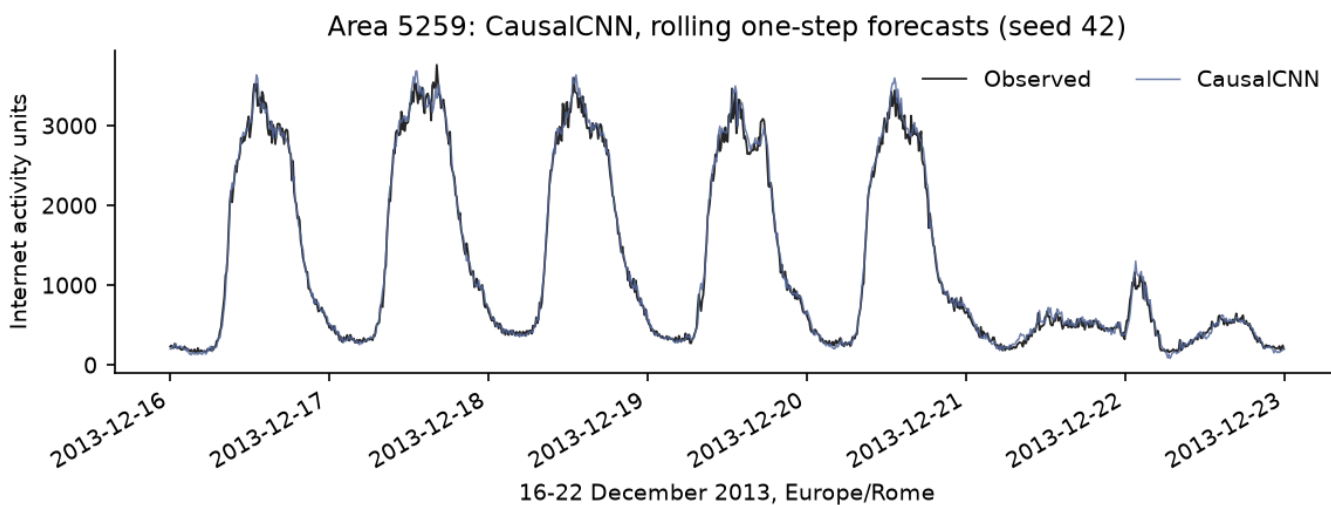


Figure 8c. CausalCNN, area 5259.

8. Discussion

14 of the 15 area-model fits beat persistence at the reference seed. The exception is area 4556 LSTM at 39.76 against persistence 39.62, which is a tie rather than a loss but is worth naming. Area 5161: CausalCNN, 12.4% below persistence. Area 5059: RidgeAR, 12.9% below persistence. Area 5259: RidgeAR, 15.0% below persistence. Area 4159: RidgeAR, 7.7% below persistence. Area 4556: RidgeAR, 10.2% below persistence.

Repeated seeds on area 5161 give: CausalCNN: mean RMSE 118.49 (SD 2.91). LSTM: mean RMSE 120.12 (SD 1.93). RidgeAR: mean RMSE 124.18 (SD 0.00). That spread is the right context for the differences between the two neural models, which are smaller than it. Average neural training costs 498 times as long for the LSTM and 368 times as long for the CNN as RidgeAR on this machine, against 1169 and 4785 parameters versus 145. Given how strong the 10-minute dependence is, there is not much room left above persistence for a more expressive model to claim at this horizon.

The daily-seasonal baseline is much worse than plain persistence everywhere, which is the clearest result in the study. Area 5161 has a daily-lag correlation of 0.894 and yesterday's value is still a bad forecast for ten minutes ahead. Strong periodicity tells you about shape, not about level at short range.

8.1 Validation and test do not agree

Area	Validation best	Test best	RidgeAR val/test	LSTM val/test	CausalCNN val/test
5161	RidgeAR	CausalCNN	152.39 / 124.18	160.26 / 118.42	160.09 / 118.16
5059	CausalCNN	RidgeAR	123.98 / 99.66	137.68 / 106.22	121.66 / 103.49
5259	CausalCNN	RidgeAR	115.23 / 93.17	116.20 / 95.02	102.15 / 93.61
4159	LSTM	RidgeAR	34.77 / 19.88	33.54 / 20.28	33.68 / 21.05
4556	CausalCNN	RidgeAR	46.32 / 35.57	47.06 / 39.76	46.07 / 36.42

The two weeks disagree about the winner in 5 of 5 areas. This is the result I trust least in the whole study and the one I would chase first. On area 5161 RidgeAR is clearly ahead on the validation week and clearly behind on the test week, and the gap in each direction is larger than the seed spread, so it is not optimization noise. With one tuning week and one test week there is no way to tell from this experiment whether the test week or the validation week is the unusual one. It does mean the headline in Section 2 should be read as "what happened on 16-22 December", not as a ranking of the three architectures.

8.2 All three models shrink toward the recent level

Model (area 5161)	Lowest-decile bias	Highest-decile bias	Highest-decile MAE
RidgeAR	+16.4	-80.8	164.4
LSTM	+13.3	-50.3	151.8
CausalCNN	+4.3	-36.2	152.1

Splitting the test errors by actual traffic decile shows the same shape almost everywhere: 14 of 15 area-model combinations under-forecast the busiest decile, 13 of 15 over-forecast the quietest, and the worst case is -80.8. So the models mostly shrink toward the recent level, compressing both ends of the range. That follows from what they are trained to do. The target is a correction to persistence, the correction is fitted to minimize squared error across mostly ordinary intervals, and the result is a conservative estimate that lags any sharp move in either direction. It is also why the peaks, which are the intervals a capacity decision would actually care about, are the ones the models handle worst.

8.3 Failure case

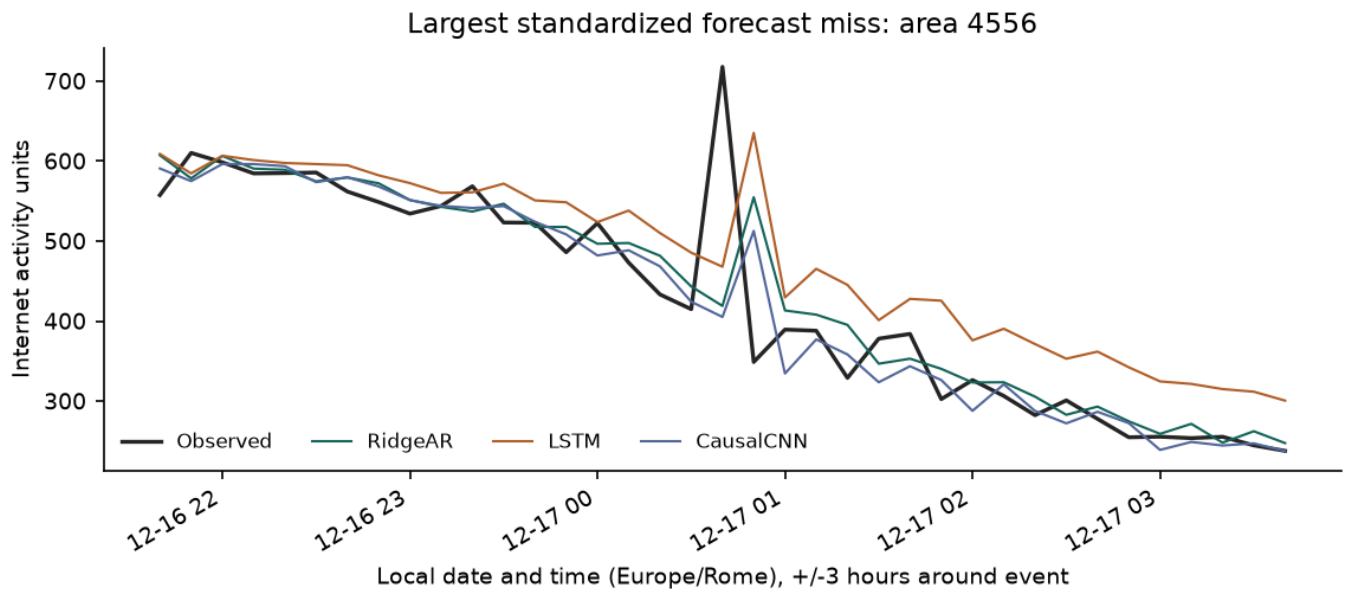


Figure 9. The largest seed-42 miss after scaling each area's error by its training standard deviation. Chosen after the models were frozen.

The worst standardized miss is area 4556 at 2013-12-17 00:40 CET. CausalCNN predicts 405.11 against an observed 717.77, an error of 312.66, or 1.24 training standard deviations. Activity goes 414.87 then 717.77 then 348.86: a single interval spike with nothing in the preceding history that points to it.

Two things are worth taking from this. First, the miss is structural rather than a tuning failure, for the reason given in Section 5 - with a persistence-correction target the loss-minimizing response to an unpredictable one-interval jump is to predict roughly no change. Second, look at the step after the spike. All three models over-predict it (RidgeAR 555, LSTM 635, CausalCNN 513 against an actual of 349), because the spike is now in their input and the persistence anchor carries it forward. So a single outlier costs two errors, not one. A univariate model has no event context and no neighbouring-area signal, so I cannot say what caused the spike, and with one interval involved I cannot rule out a measurement artifact either.

9. Conclusion and future work

On the busiest area, CausalCNN gives the lowest reference-seed RMSE at 118.16 against persistence at 134.88. 14 of the 15 area-model fits beat persistence, the one exception being a tie. The daily-seasonal baseline is far behind everywhere, which says the useful signal at a 10-minute horizon is recent level rather than daily shape.

I am less confident about the ranking between the three models than about that. Validation and test disagree on 5 of 5 areas, the differences between the two neural models are inside the seed spread, and RidgeAR wins on validation in every tuning round while losing on test for the busiest area. What the evidence does support is that a 145-parameter linear model is a serious competitor here, and that anything more expensive needs to show a gain that survives more than one week before it is worth the training cost.

The main limitations are one held-out week, five areas chosen using full-period totals, hyperparameters tuned on a single area and single seed, and three seeds that measure optimization variance rather than uncertainty about future weeks. The epoch cap in the first three rounds is a concrete example of how a constraint set on one area can quietly distort the others, and I only caught it by auditing `epochs_run` against the budget. Next steps I would actually run: rolling-origin evaluation across several weeks so the ranking question can be settled, area selection made prospectively from a period before the evaluation window, matched compute budgets per model, trimming the CNN dilations to the input length, and adding neighbouring-area inputs to see whether spikes like the one in Section 8.3 are predictable from space even when they are not predictable from time.

Appendix A. Sensitivity across seeds

Area	Model	RMSE mean (3 seeds)	RMSE sample SD
4159	CausalCNN	20.15	0.78
4159	LSTM	20.72	1.26
4159	RidgeAR	19.88	0.00
4556	CausalCNN	37.65	1.15
4556	LSTM	42.08	2.01
4556	RidgeAR	35.57	0.00
5059	CausalCNN	103.68	2.58
5059	LSTM	101.82	4.43
5059	RidgeAR	99.66	0.00
5161	CausalCNN	118.49	2.91
5161	LSTM	120.12	1.93
5161	RidgeAR	124.18	0.00
5259	CausalCNN	91.18	2.28
5259	LSTM	96.05	1.67
5259	RidgeAR	93.17	0.00

These standard deviations cover seeds 42, 43, 44 and describe how much the optimizer moves, nothing else. They are not confidence intervals and they say nothing about a different test week or a different area. Model-specific plots for squares 4159 and 4556 are in `results/figures/` alongside the nine primary-area plots, and every per-timestamp prediction is kept so the numbers can be rechecked independently.

References

- [1] G. Barlacchi et al., "A multi-source dataset of urban life in the city of Milan and the Province of Trentino," *Scientific Data*, vol. 2, art. 150055, 2015, doi: 10.1038/sdata.2015.55. <https://doi.org/10.1038/sdata.2015.55>.
- [2] A. Azari, P. Papapetrou, S. Denic, and G. Peters, "Cellular Traffic Prediction and Classification: a comparative evaluation of LSTM and ARIMA," *arXiv:1906.00939*, 2019. <https://arxiv.org/abs/1906.00939>.
- [3] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv:1803.01271*, 2018. <https://arxiv.org/abs/1803.01271>.
- [4] C. Zhang and P. Patras, "Long-Term Mobile Traffic Forecasting Using Deep Spatio-Temporal Neural Networks," *arXiv:1712.08083*, 2017. <https://arxiv.org/abs/1712.08083>.
- [5] Telecom Italia, "Telecommunications - SMS, Call, Internet - MI," *Harvard Dataverse*, 2015. <https://doi.org/10.7910/DVN/EGZHFV>. Data under ODbL 1.0; from [BigDataChallenge contest](#).
- [6] Source code and reproducibility materials: [GitHub repository](#).

[7] Demo video: [PASTE_VIDEO_LINK_HERE](#)